Restaurant Ordering System

Final Project Report

Introduction

The Restaurant Ordering System is a Java desktop application that supports menu families, meals, matching drinks, extra toppings, discounts, checkout, kitchen progress and customer notifications. Six design patterns operate within a single transaction: Factory Method, Abstract Factory, Decorator, Observer, Strategy and Facade.

The application provides an order-entry screen, a sortable order table, a receipt view, separate kitchen and customer notification feeds, and an event log. Order records are saved to an XML file and restored when the application restarts.

Pattern	Implementation	Role in the transaction
Factory Method	MealFactory and concrete meal creators	Create a specific meal through an overridable creation method.
Abstract Factory	MenuFactory, RegularMenuFactory, VegetarianMenuFactory	Create related meal and drink products for the selected family.
Decorator	MealDecorator, ExtraCheese, Mushrooms, ExtraSauce	Wrap a meal to extend its description and price.
Observer	OrderSubject, OrderEventSource, OrderObserver	Notify KitchenDisplay and CustomerDisplay after saved changes.
Strategy	DiscountStrategy and three implementations	Choose no discount, student 10% or loyalty 15%.
Facade	RestaurantFacade	Coordinate menu creation, pricing, orders, persistence and notifications.

Java 17 is the compilation target. The desktop interface uses Swing, persistence uses the Java XML Properties API, and tests use JUnit 5. There are no external runtime libraries. All customer names and transactions shown in the report are application sample data.

Body Meal Families and Toppings

Abstract Factory

MenuFactory declares two product methods: createMeal and createDrink. RegularMenuFactory supplies regular meals and cola. VegetarianMenuFactory supplies vegetarian meals and fresh lemonade. The family selector chooses one factory for both products. A drink is optional, and when selected its quantity matches the meal quantity.

```
public interface MenuFactory {
    Meal createMeal(MealType type);
    Drink createDrink();
}
```

Factory Method

Each menu family delegates meal creation to a MealFactory selected by MealType. PizzaFactory, BurgerFactory and PastaFactory create regular meals, while VegetarianMealFactory creates a vegetarian variant. Factory Method creates an individual meal; Abstract Factory supplies the family of meal and drink products.

```
public final class PizzaFactory extends MealFactory {
    @Override
    public Meal createMeal() { return new Pizza(); }
}
```

Decorator

MealDecorator implements Meal and holds another Meal. Each topping delegates to the wrapped meal and adds its own price and description. Several decorators can be stacked without changing the original Pizza, Burger or Pasta classes.

```
Meal base = menu.createMeal(MealType.PIZZA);
Meal withToppings = new Mushrooms(new ExtraCheese(base));
long price = withToppings.priceCents();
```

For a regular pizza, the base is 1,200 cents, extra cheese adds 150 cents and mushrooms add 100 cents. The resulting meal costs $14.50. A vegetarian burger and fresh lemonade cost $9.50 and $2.50 respectively, for a $12.00 basket before discounts.

Body Pricing Orders and Notifications

Strategy

DiscountStrategy separates discount calculation from checkout. NoDiscount returns zero; StudentDiscount returns 10 percent; LoyaltyDiscount returns 15 percent. The selected strategy is used both for the basket preview and the saved order. Integer cents avoid floating-point accumulation. Fractional discount cents are rounded down.

```
long discount = strategy(discountName).discountCents(subtotal);
```

Two decorated pizzas cost $29.00, a burger with sauce costs $9.50 and a cola costs $2.00. The subtotal is $40.50; the student discount is $4.05; the final total is $36.45.

Observer

OrderEventSource stores observers in a CopyOnWriteArrayList. KitchenDisplay and CustomerDisplay subscribe through OrderSubject. Once an order is saved, the facade publishes an immutable OrderEvent. The kitchen receives meal details and progress; the customer receives a readable status message. Duplicate subscriptions are prevented, and a listener can unsubscribe from its own callback. A failing listener is recorded without stopping other deliveries.

```
events.subscribe(kitchen);
events.subscribe(customers);
events.publish(new OrderEvent(order, previousStatus, clock.instant()));
```

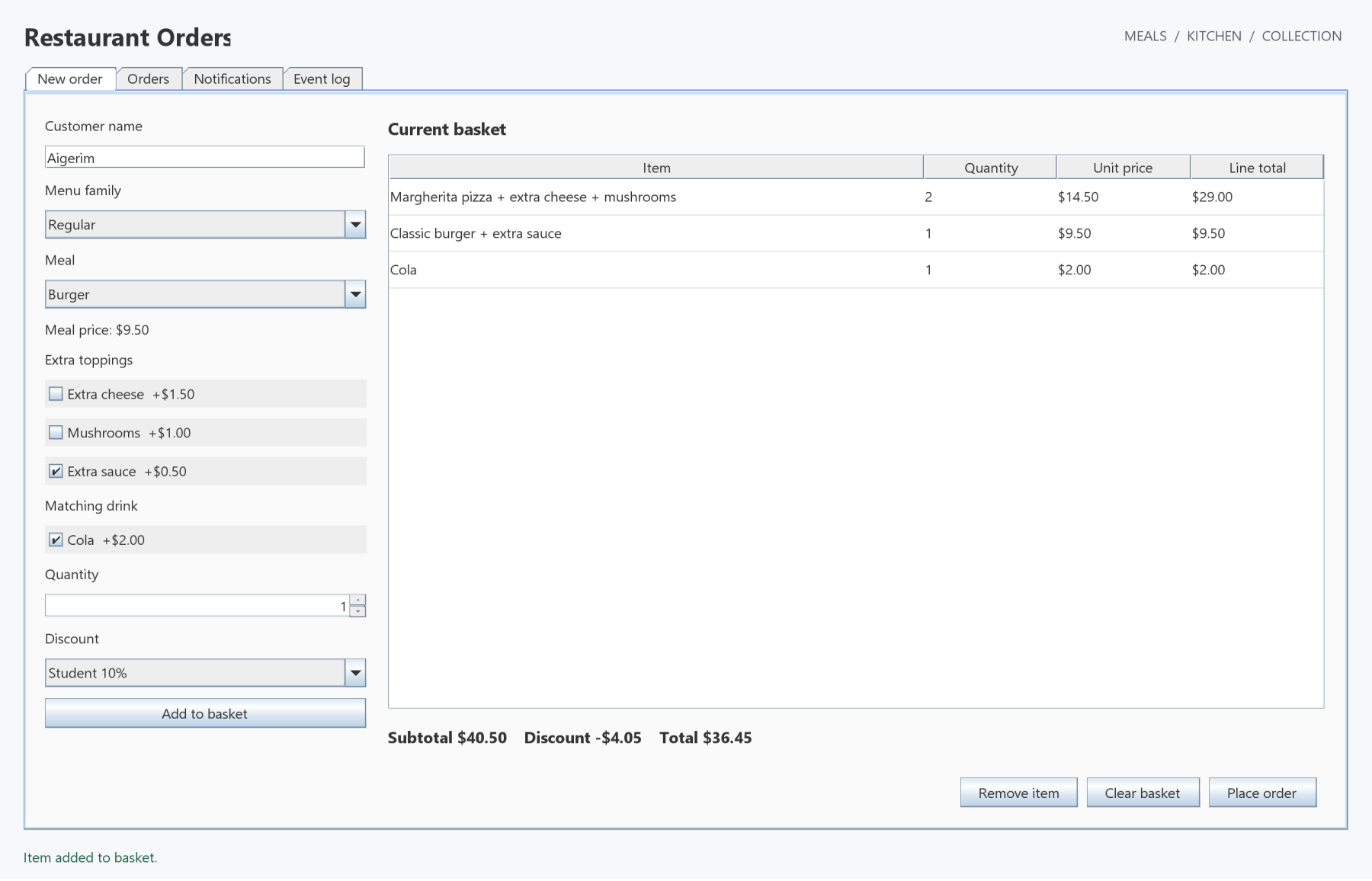
Facade

RestaurantFacade is the entry point for menu, pricing and order operations. It selects the menu factory, applies decorators and discount strategies, validates orders, saves changes through OrderRepository and publishes notifications. The order-entry and status buttons call the facade directly.

```
Order updated = previous.withStatus(next);
repository.save(updated);
events.publish(new OrderEvent(
    updated, previous.status(), clock.instant()));
```

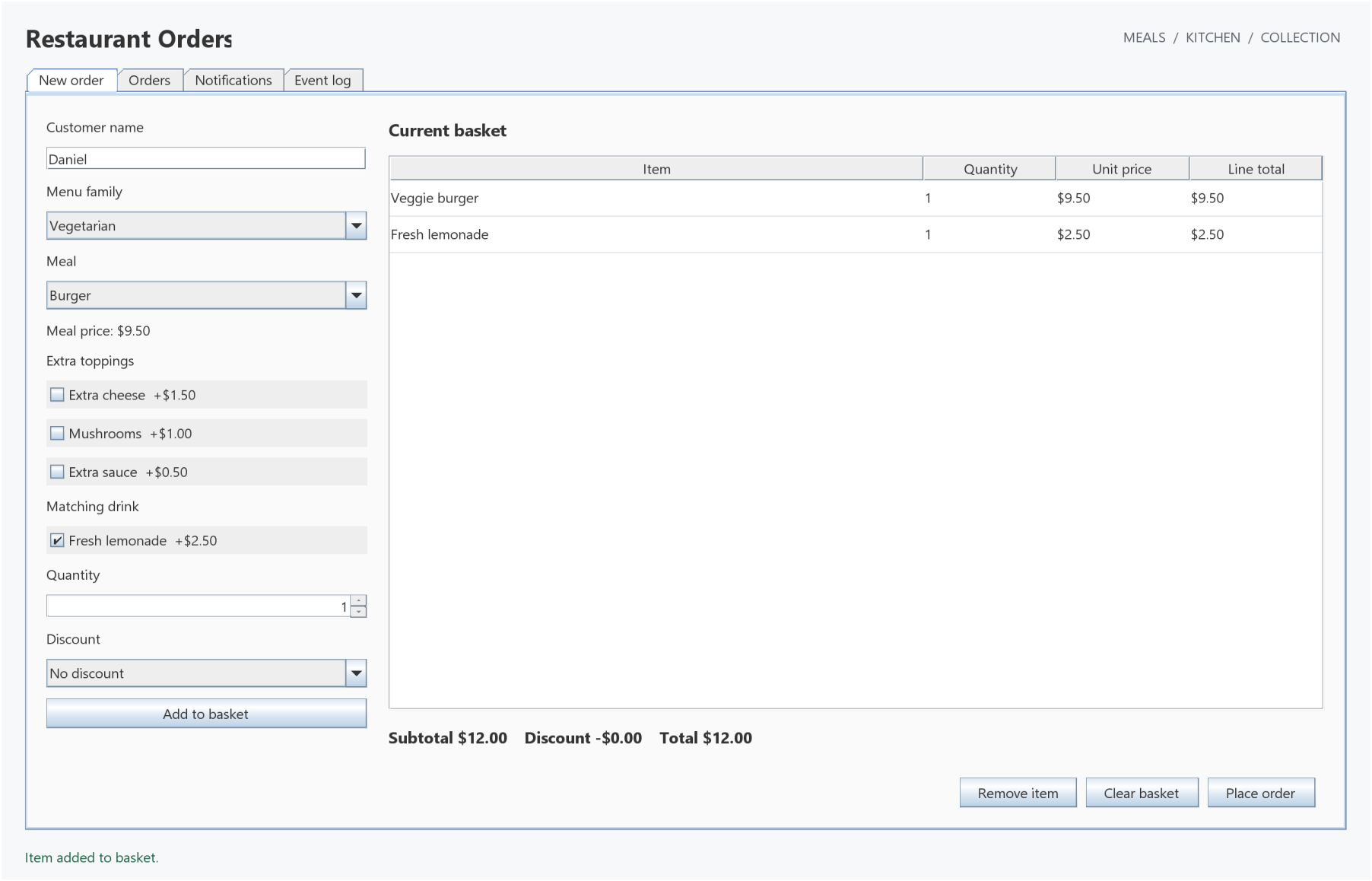
The normal lifecycle is Placed, Preparing, Ready, Completed. Cancellation is allowed while Placed or Preparing. Completed and Cancelled are terminal states. An invalid change does not write a new status or notify observers. Notifications and the event log are session data; saved receipts and statuses survive restart.

Application Regular Menu



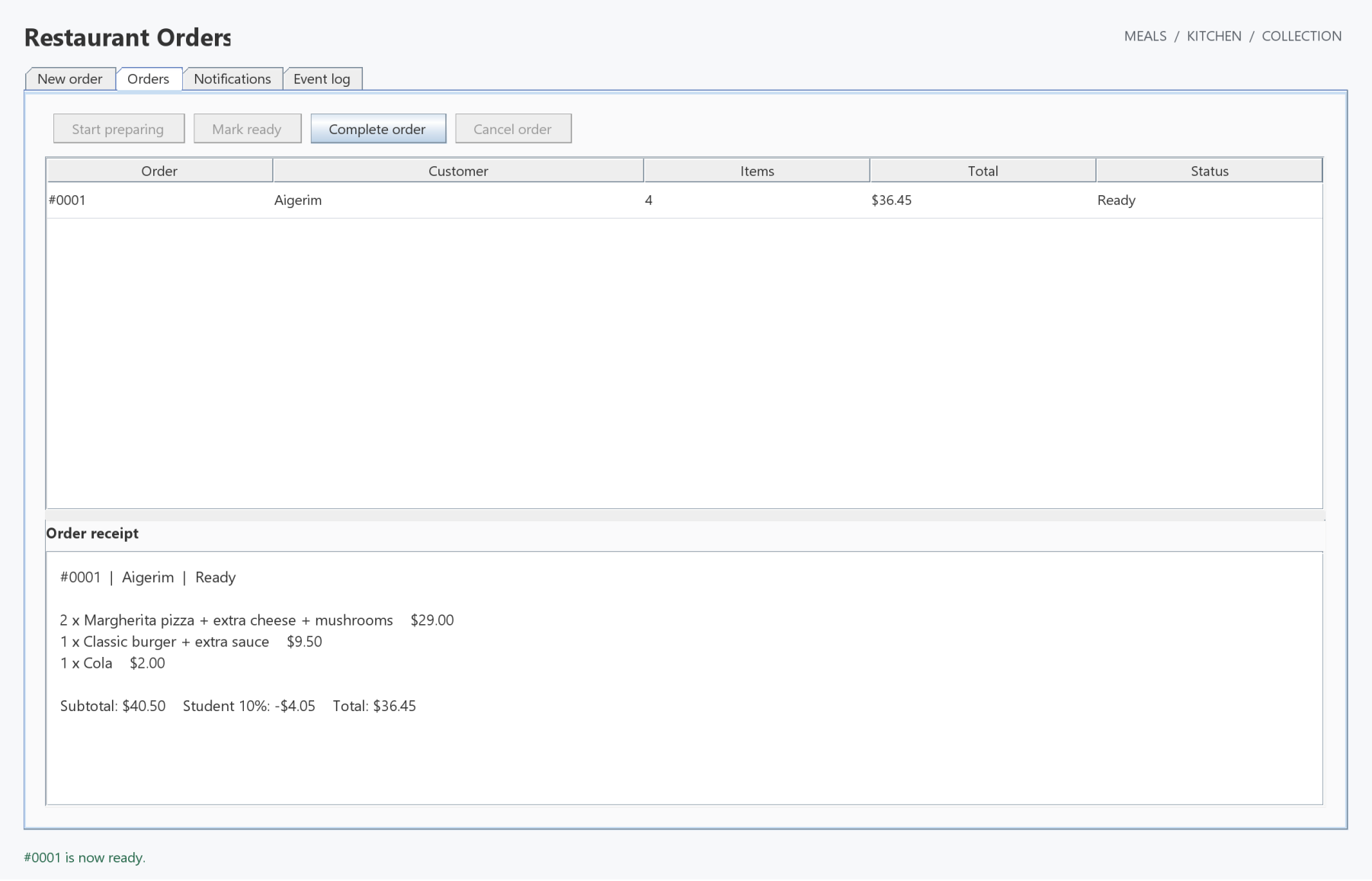
The regular family supplies regular meals and cola. The basket combines decorated meals, a drink and a selected discount. The same pricing logic produces the $36.45 total during checkout.

Application Vegetarian Menu



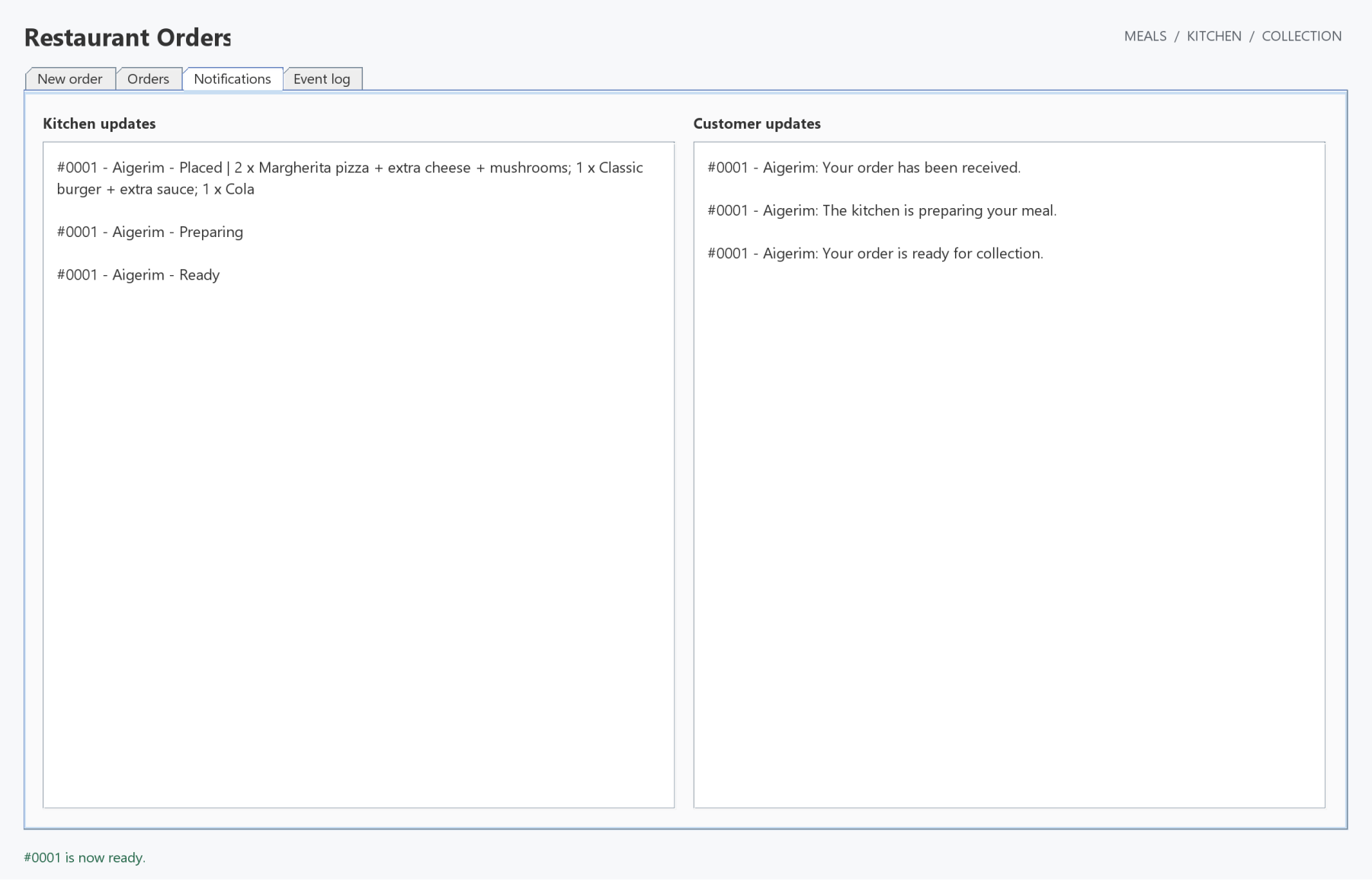
Changing the family switches both products. The vegetarian burger and fresh lemonade appear as separate basket lines, with a combined price of $12.00.

Application Order Tracking



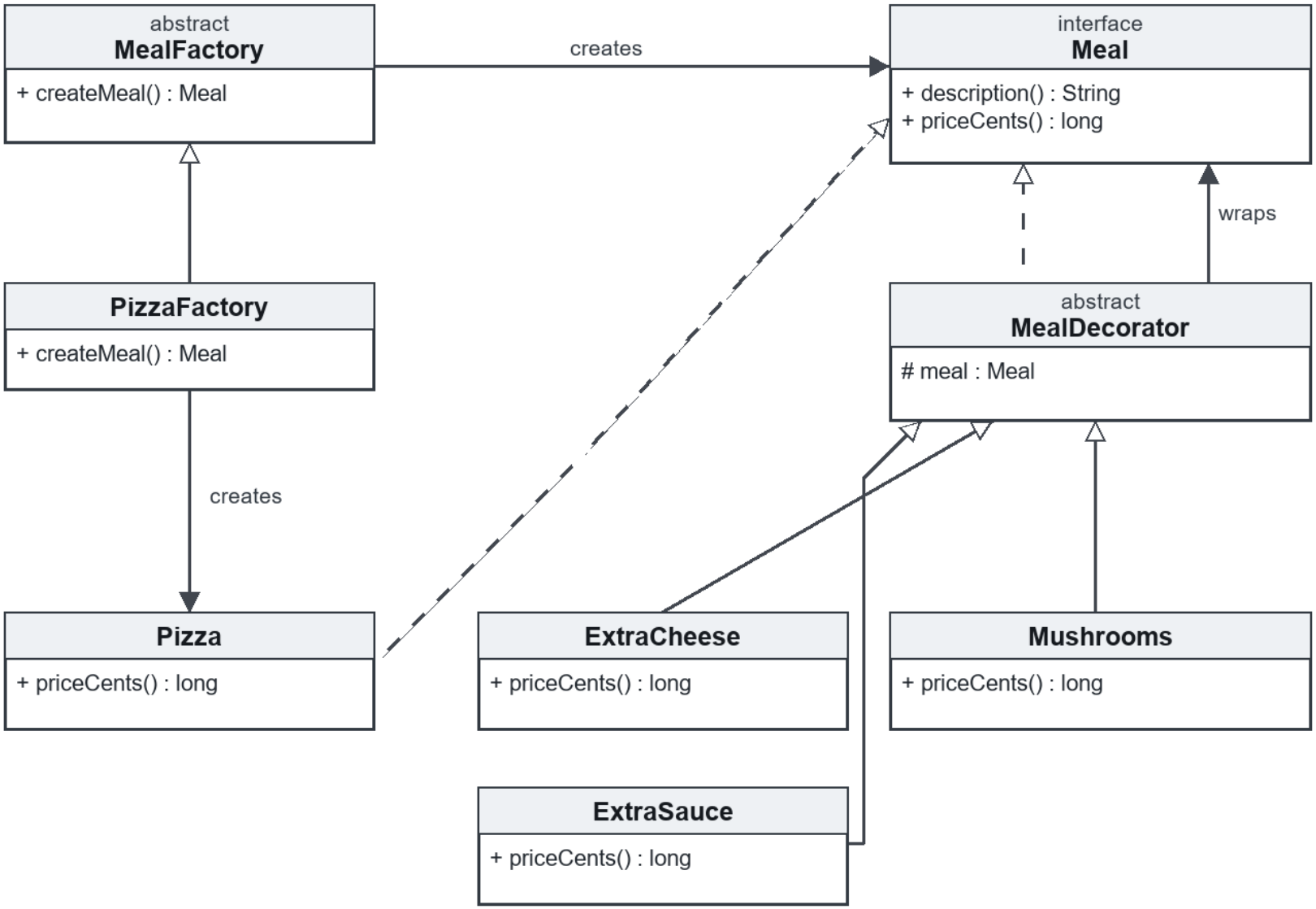
A saved order is shown at Ready. Only valid next actions are enabled. The receipt preserves the descriptions, quantities, prices and discount used at checkout.

Application Notifications



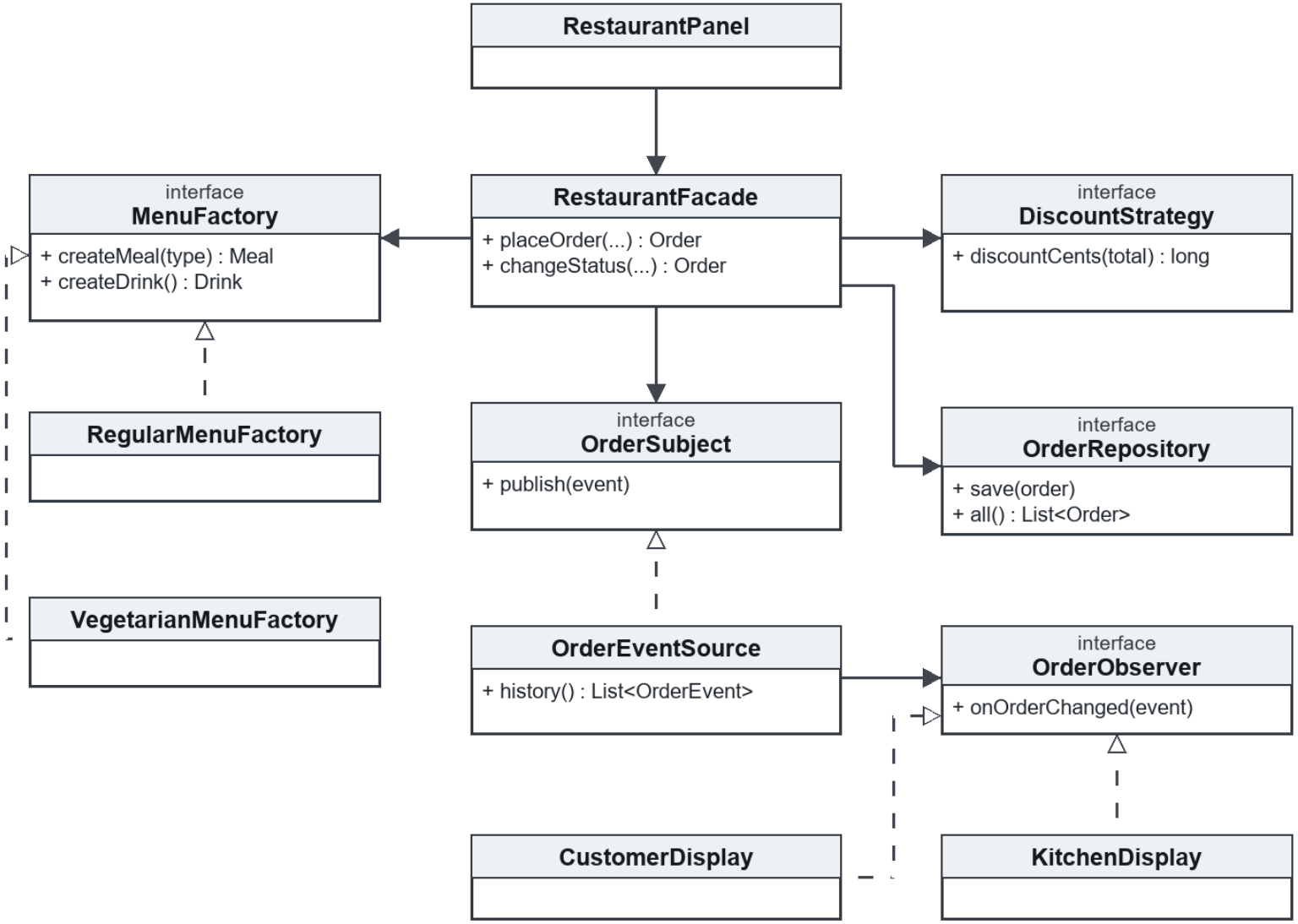
The same status events reach separate kitchen and customer observers. Kitchen messages include the ordered meals; customer messages describe the current progress.

UML Factory Method and Decorator



Selected creation and topping classes. Hollow triangles indicate inheritance or interface implementation; ordinary arrows show creation or delegation. BurgerFactory and PastaFactory follow the same creator structure as PizzaFactory.

UML Ordering and Notification Workflow



The panel calls the facade, which selects a menu family and discount strategy, saves orders through the repository and publishes through OrderSubject. The event source notifies the kitchen and customer observers. Class, sequence and state diagrams are included as PlantUML sources.

Testing Results and Conclusion

Testing Results

Test suite	Tests	Failures and errors
ApplicationViewTest	4	0
MealPatternTest	16	0
MenuFamilyTest	6	0
OrderObserverTest	4	0
OrderWorkflowTest	15	0
RepositoryTest	4	0
Total	49	0

The tests cover both menu families, factory products, decorator composition, discount arithmetic, order validation, status transitions, observer behaviour, immutable basket snapshots, XML restart recovery and failed writes. Desktop tests execute checkout and status buttons, verify sorted-row selection and confirm that vegetarian meals and drinks reach the basket together.

The integrated checkout test verifies the $40.50 subtotal, $4.05 discount and $36.45 total. Four lifecycle events reach both kitchen and customer observers. Persistence tests confirm that special characters and paired products survive XML round trips. The project was verified using JDK 27 with release 17 targeting Java 17 APIs and bytecode.

Conclusion

The application implements six distinct design patterns within a working restaurant ordering workflow. Menu families, individual meal creation, toppings, pricing, notifications and order coordination remain separate responsibilities. The desktop interface uses the same validated services as the automated tests, and saved orders preserve the original receipt values and latest status.

Further Work

A future version could add payment-provider integration, table reservations, inventory availability and role-based staff access. A transactional database would support several ordering terminals sharing one history. Notification retention and external SMS delivery could extend the current session-based kitchen and customer feeds. These extensions are outside the implemented single-terminal application.